

UG-CMS — Uganda Case Management System

A comprehensive digital platform for managing the entire case lifecycle — from crime reporting to final appeal — across all courts in Uganda's judiciary system.

Tech Stack

Layer	Technology
Framework	Next.js 16 (App Router, React 19)
Runtime	Bun
Language	TypeScript (strict mode)
Database	PostgreSQL 16 (via Prisma ORM v7)
Adapter	<code>@prisma/adapter-pg</code> (direct connection pooling)
Auth	JWT (access + refresh tokens), bcrypt password hashing, cookie-based sessions
Styling	Tailwind CSS v4, Radix UI primitives
Forms	React Hook Form + Zod validation
Charts	Recharts
Icons	Lucide React
Containerization	Docker Compose (PostgreSQL + pgAdmin)

Architecture

```

src/
├── app/                                # Next.js App Router (file-based routing)
│   ├── (auth)/login/                  # Public authentication pages
│   ├── (dashboard)/dashboard/         # Protected dashboard pages (route group)
│   │   ├── cases/                    # Case management CRUD
│   │   ├── hearings/                 # Hearing scheduling & tracking
│   │   ├── evidence/                 # Evidence custody chain
│   │   ├── investigations/           # Police/Directorate investigations
│   │   ├── bail/                     # Bail applications & sureties
│   │   ├── appeals/                  # Appeal management
│   │   ├── detention/                # Detention & prisoner tracking
│   │   ├── courts/                  # Court registry
│   │   ├── users/                    # User management
│   │   ├── roles/                    # RBAC role definitions
│   │   ├── documents/                # Document management
│   │   ├── reports/                  # Analytics & reporting
│   │   ├── audit-logs/               # Audit trail viewer
│   │   ├── notifications/            # Notification center
│   │   └── settings/                 # System settings
│   └── api/                           # REST API routes (Next.js route handlers)
│       ├── auth/                     # login, register, me, logout, refresh
│       ├── cases/                    # CRUD with RBAC enforcement
│       ├── hearings/
│       ├── evidence/
│       ├── investigations/
│       ├── bail/
│       ├── appeals/
│       ├── courts/
│       ├── detention/
│       ├── documents/
│       ├── users/
│       ├── roles/
│       ├── permissions/
│       ├── audit-logs/
│       ├── notifications/
│       ├── reports/                  # Dashboard analytics
│       ├── sessions/
│       └── system/
├── components/
│   ├── layout/                        # AppShell, Sidebar, Header, Breadcrumbs
│   ├── ui/                            # Reusable primitives (Button, Card, Input,
│   │   Badge, DataTable, Skeleton)
│   └── shared/                        # Cross-cutting components (EmptyState,
│   │   PageHeader)
├── config/
│   └── navigation.ts                  # Sidebar navigation tree with role-based
│   │   visibility
├── lib/
│   ├── auth/
│   │   └── auth.ts                   # JWT sign/verify, bcrypt hashing

```

```

| | | └─ session.ts           # Cookie-based session management
| | |   └─ rbac.ts           # Role-based access control (permission checks)
| | | └─ prisma/
| | |   └─ client.ts         # Singleton Prisma client with Pg adapter
| | |   └─ utils/
| | |     └─ response.ts     # Standardized API response helpers
| | |     └─ pagination.ts   # Cursor/offset pagination
| | |     └─ audit.ts        # Audit log writer
| | |     └─ case-number.ts  # Case number generation (court-
code/type/seq/year)
| | |       └─ cn.ts         # Tailwind class merge utility
| | |       └─ constants.ts  # App-wide constants
| └─ providers/
|   └─ auth-provider.tsx    # Client-side auth context (login, logout, user
state)
|   └─ theme-provider.tsx   # Dark/light theme via next-themes
|   └─ toast-provider.tsx   # Toast notification provider (Sonner)
└─ types/
    └─ index.ts             # Shared TypeScript types
└─ middleware/
    └─ (via proxy.ts)       # Request-level auth gateway

```

Request Flow (Auth)



Database Domains

The schema models 10 interconnected domains:

Domain	Tables	Description
User & RBAC	users , roles , user_roles , permissions , role_permissions	Authentication, authorization, role hierarchy with court-scoped assignments
Organization	courts , police_stations	Court hierarchy (9 levels, self-referencing), police station registry
Case Management	cases , criminal_case_details , civil_case_details , case_parties , case_timelines	Full case lifecycle with 15 statuses, 6 case types, criminal/civil subtypes
Court Operations	court_sessions , hearings	Session management, 15 hearing types, adjournment tracking
Investigation & Evidence	investigations , evidence , evidence_chains	Investigation workflow, physical/digital evidence, custody chain
Bail & Surety	bail_applications , sureties	Bail lifecycle (granted/denied/revoked), surety verification
Appeal	appeals , appeal_decisions	Multi-level appeals (first/second/constitutional), 6 outcomes
Documents	documents , document_tags	Versioned document management with access levels and tags
Detention	detention_records , prison_transfers	Prisoner tracking, facility transfers, release management
Notifications	notifications , notification_preferences	Multi-channel (email/SMS/in-app/system), read receipts
Audit	audit_logs	Immutable audit trail with old/new value snapshots

Case Lifecycle (15 Statuses)



RBAC Model

- **Permissions** are `ResourceType × PermissionAction` tuples (e.g., `CASE:CREATE`, `EVIDENCE:READ`)
- **Roles** are named collections of permissions with a `hierarchy` level
- **UserRole** assignments can be court-scoped (a judge assigned to a specific court)
- **system_administrator** bypasses all permission checks (full access)
- 11 action types: `CREATE`, `READ`, `UPDATE`, `DELETE`, `APPROVE`, `REJECT`, `ASSIGN`, `TRANSFER`, `ARCHIVE`, `EXPORT`
- 16 resource types: `CASE`, `USER`, `ROLE`, `COURT`, `HEARING`, `EVIDENCE`, `DOCUMENT`, `BAIL_APPLICATION`, `APPEAL`, `DETENTION`, `INVESTIGATION`, `NOTIFICATION`, `AUDIT_LOG`, `REPORT`, `SETTINGS`

API Design

All API routes follow a consistent pattern:

GET	/api/cases	→ list with filtering, pagination, RBAC-scoped data
POST	/api/cases	→ create with Zod validation + audit log
GET	/api/cases/[id]	→ single resource
PUT	/api/cases/[id]	→ update (checking record ownership or permission)
DELETE	/api/cases/[id]	→ soft-delete or archive

Response envelope:

```
{
  success: true,
  data: T | T[],
  pagination?: { total, page, limit, totalPages }
}
// or
{ success: false, error: "message" }
```

Getting Started

Prerequisites

- [Bun](#) $\geq 1.x$
- [Docker](#) & Docker Compose (for PostgreSQL)

Quick Start

```
# 1. Clone and install
git clone <repo-url>
cd UG-CMS
bun install

# 2. Copy environment file
cp .env.example .env # or create .env with DATABASE_URL

# 3. Start database
bun run docker:up

# 4. Initialize database
bunx prisma generate
bunx prisma db push
bun run db:seed

# 5. Start dev server
bun run dev
```

Open <http://localhost:3000>. Login with:

- **Admin:** `admin@ugcms.gov` / `admin123`
- **All users:** password is `admin123`

Environment Variables

```
DATABASE_URL="postgresql://ugcms:ugcms_dev_pass@localhost:5433/ug_cms"
JWT_SECRET="your-jwt-secret" # default: "dev-secret"
JWT_REFRESH_SECRET="your-refresh-secret" # default: "dev-refresh-secret"
```

Available Scripts

Script	Description
<code>bun run dev</code>	Start dev server with Turbopack
<code>bun run build</code>	Production build
<code>bun run start</code>	Start production server
<code>bun run lint</code>	Run ESLint
<code>bun run db:generate</code>	Generate Prisma client
<code>bun run db:push</code>	Push schema to database
<code>bun run db:migrate</code>	Run Prisma migrations
<code>bun run db:studio</code>	Open Prisma Studio (DB GUI)
<code>bun run db:seed</code>	Seed database with demo data
<code>bun run docker:up</code>	Start PostgreSQL + pgAdmin
<code>bun run docker:down</code>	Stop containers
<code>bun run setup</code>	Full setup: docker up → generate → push → seed

Docker Services

Service	Port	Credentials
PostgreSQL 16	5433	ugcms / ugcms_dev_pass
pgAdmin	5051	admin@ugcms.gov / admin

Production Notes

- DB Migrations:** Use `prisma migrate dev` for development, `prisma migrate deploy` in CI/CD
- JWT Secrets:** Always set strong `JWT_SECRET` and `JWT_REFRESH_SECRET` in production

3. **Cookies:** `secure: true` is automatically enabled when `NODE_ENV=production`
 4. **Middleware:** `proxy.ts` handles all auth gating — add new public routes to the `PUBLIC_PATHS` array
 5. **File Uploads:** The `uploads/` directory stores document files; mount a persistent volume in production
 6. **Session Duration:** Access tokens expire in 15 minutes; refresh tokens in 7 days
-